00 · Start here: the trainer kit, wiring discipline, and the whole lab

ECE2131 Digital Electronics Lab · source: the lab manual (2025), 55 pp. · every design in this pack is machine-checked over all inputs (src/verify.py, 49 checks pass) · marks: 100 = internal 60 + external 40; exam 3 hours

The lab as one path

1 Gates & universality → 2 Adders/subtractors → 3 NAND/NOR design → 4 Code converters →
5 Comparator & parity → 6–8 MUX, DEMUX, decoder → 9–10 Flip-flops →
11–12 Counters & shift registers → 13 FSM → 14 TTL

- **Experiments 1–8 are combinational:** the output depends only on the present inputs.
- **Experiments 9–13 are sequential:** they add memory (flip-flops), so the output also depends on the past.
- **Exp 14** looks inside one gate.

You already know the theory (ECE2102). What this pack adds is the part the exam tests at the bench: **which IC, which pin, which wire, what the LEDs should show, and why.**

The IC trainer kit

Part of the kit	Use
+5 V and GND terminals	power every IC: **+5 V only. More destroys TTL.**
logic input switches	give 0/1 to inputs (up = 1)
LED indicators	show outputs (lit = 1)
clock generator (≈ 1 Hz)	slow continuous clock for counters
pulser / mono-pulse (debounced)	one clean clock edge per press. Use this for flip-flop tables.
breadboard	5-hole vertical strips are one node; the long rails are the power buses

Wiring discipline (the manual's "common causes of problems", made into a routine)

EVERY CIRCUIT, EVERY TIME

1. Power **off**.
2. Plug each IC across the breadboard's centre gap, with the **notch to the left** (or up) and all ICs facing the same way. Pin 1 is next to the notch or dot. Pins count **anticlockwise** seen from the top.
3. Wire **VCC and GND of every IC first**: - 14-pin gates: VCC = 14, GND = 7. - 16-pin MSI chips: VCC = 16, GND = 8. - **Exceptions: 7483 (VCC 5, GND 12) and 7473 (VCC 4, GND 11).**
4. Wire short connections before long ones, and tick each one off on the circuit diagram as you go.
5. Tie every unused input. Unused AND/NAND inputs go to an input you are using or to +5 V; unused OR/NOR inputs go to GND. A floating TTL input reads as HIGH, which silently forces a NOR output to 0.
6. Have a partner check the wiring. Then power **on**, and walk the truth table row by row.
7. Never drive one gate input from two outputs. Never rewire with the power on.

WHEN NOTHING WORKS

In this order:

1. Is the IC powered? Check with the meter on the IC's own VCC/GND pins.
2. Is the IC in the right way round?
3. Is the IC dead? Swap it.
4. Is the LED wired to the output pin or to an input pin?

Most "wrong" truth tables are a missing ground or an IC in backwards.

How each chapter works

Each experiment gives you, in order:

1. the aim as the question;
2. a guess-first line;
3. the design (expressions, already checked);
4. the **pin-to-pin wiring table**;
5. what the LEDs should show;
6. a ready "write in your record" block;
7. traps and a check, with answers at the end of the chapter.

The pin diagram of every IC appears where it is first used.

Errors in the manual

Where	The manual says	Correct
Exp 1, NOT gate	truth table A = 0 → Y = 0, then 0 → 1	A = 0 → Y = 1; A = 1 → Y = 0
IC table	74138 listed as "negative edge triggered dual J-K F/F"	74138 is a 3:8 decoder (the 7473 is the JK)
Exp 2, Fig 2.5	"Pin Diagram of IC 7485 (4-bit binary full adder)"	the adder is the **7483** (the 7485 is the comparator)
Exp 2, subtractor	"borrow output at C_{out}"	for A − B, C_{out} = 1 means **no** borrow (A ≥ B); C_{out} = 0 means the answer is negative, in 2's complement
Exp 4, Table 4.2	"BCD to grey code"	BCD to **Excess-3**
Exp 7	aim says "multiplexer"; parts 74151, 74153	demultiplexer; the 1:8 needs the **74138**
Exp 13	the list says detect **101**; the experiment designs **1010**	this pack gives both
Exp 13 (ii)	$F^+ = XY + FY''$	$F^+ = XY + FY'$
Exp 11, ring counter	"clear all flip-flops, then clock"	a ring counter cleared to 0000 stays at 0000 forever. One flip-flop must be **preset** to 1.
Exp 14	no circuit given	see Exp 14 in this pack (LTspice-verified)

Exp 1 · Logic gates, and NAND/NOR as universal gates

Manual pp. 1–4 · ICs 7400, 7402, 7404, 7408, 7432, 7486 · trainer kit

1 Gates & universality → 2 Adders/subtractors → 3 NAND/NOR design → 4 Code converters

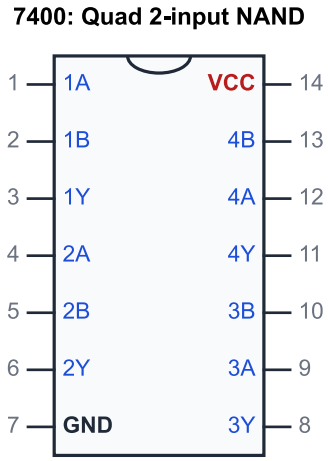
MANUAL EXP 1, AIM

Verify the truth tables of AND, OR, NOT, NAND, NOR and XOR; realise NOT, AND and OR using only NAND gates, then only NOR gates. Viva form: "Why are NAND and NOR called universal? Show OR from NANDs only."

GUESS FIRST

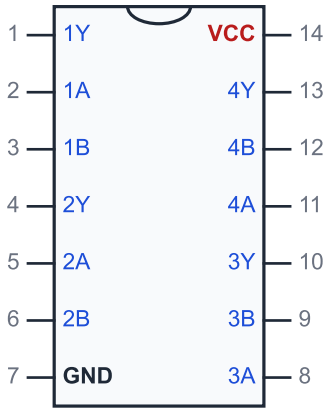
How many 2-input NANDs make an OR? Sketch it before reading on.

The six ICs



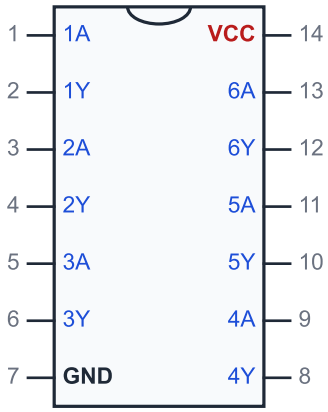
7408 (AND), 7432 (OR) and 7486 (XOR) use exactly this layout. Only the gate inside changes.

7402: Quad 2-input NOR



The 7402 is different: each output comes FIRST (pins 1, 4, 10, 13).

7404: Hex inverter



Six inverters.

Testing a gate IC (e.g. the 7408): connect A → pin 1, B → pin 2, LED ← pin 3, VCC 14, GND 7. Walk 00, 01, 10, 11. Repeat for gates 2–4 if you have time: a chip can have one dead gate.

A B	AND	OR	NAND	NOR	XOR	NOT A
0 0	0	0	1	1	0	1
0 1	0	1	1	0	1	1
1 0	0	1	1	0	1	0
1 1	1	1	0	0	0	0

Universality

A gate is **universal** if NOT, AND and OR (and so any function) can be built from it alone.

Make	From NAND (7400)	From NOR (7402)
NOT A	NAND(A, A)	NOR(A, A)
AND	NAND, then a NAND-inverter: ((AB)')'	invert both inputs, then NOR: (A' + B')' = AB (De Morgan)
OR	invert both inputs, then NAND: (A'B')' = A + B (De Morgan)	NOR, then a NOR-inverter: ((A + B)')'
gates needed	NOT 1, AND 2, OR 3	NOT 1, OR 2, AND 3

Wiring (the manual's pin numbers)

Circuit	Wires
NOT (NAND)	A → 7400 pins 1 and 2; Y = pin 3
AND (NAND)	A → 1, B → 2; pin 3 → pins 4 and 5; Y = pin 6
OR (NAND)	A → 1 and 2 (pin 3 = A'); B → 4 and 5 (pin 6 = B'); pin 3 → 9, pin 6 → 10; Y = pin 8
NOT (NOR)	A → 7402 pins 2 and 3; Y = pin 1
OR (NOR)	A → 2, B → 3; pin 1 → pins 5 and 6; Y = pin 4
AND (NOR)	A → 2 and 3 (pin 1 = A'); B → 5 and 6 (pin 4 = B'); pin 1 → 8, pin 4 → 9; Y = pin 10

WRITE IN YOUR RECORD

Result: the truth tables of AND, OR, NOT, NAND, NOR and XOR were verified. NOT, AND and OR were realised using only NAND gates (1, 2, 3 gates) and using only NOR gates (1, 3, 2 gates), and their truth tables verified.

Theory for the record: logic gates implement Boolean operations: AND (Y = AB), OR (Y = A + B), NOT (Y = A'), NAND (Y = (AB)'), NOR (Y = (A + B)'), XOR (Y = AB' + A'B). TTL ICs of the 74 series contain several gates in a 14-pin DIP, with VCC on pin 14 and GND on pin 7. NAND and NOR are universal gates: NOT, AND and OR, and therefore any Boolean function, can be built using only one of them. By De Morgan's theorems, (A'B')' = A + B and (A' + B')' = AB.

CHECK 1

(a) Make XOR from 2-input NANDs only: how many gates? (b) Why does a floating input on a 7402 NOR give a stuck-at-0 output? (c) Which pin is the output of gate 3 on the 7400, and on the 7402?

Answers

Opening question. Each of them alone can make NOT, AND and OR, and so any function. OR = NAND(NAND(A,A), NAND(B,B)): **3 NANDs**.

Check 1. (a) **4 NANDs**: n1 = (AB)', then A $\oplus$ B = NAND(NAND(A, n1), NAND(B, n1)). (b) A floating TTL input behaves as logic 1, and a NOR with any input at 1 outputs 0. (c) 7400 gate 3: inputs 9, 10 → output **8**. 7402 gate 3: inputs 8, 9 → output **10**.

Exp 2 · Half/full adder and subtractor; 7483 adder, 2's-complement subtractor, BCD adder

Manual pp. 5–9 · ICs 7486, 7408, 7432, 7404, 7483 · all expressions checked over every input

MANUAL EXP 2, AIM

Design and test a half adder/half subtractor and a full adder/full subtractor; study the 7483. Viva form: "How does a single 7483 subtract, and what does C_{out} tell you after a subtraction?"

GUESS FIRST

Subtraction A − B = A + (2's complement of B). How would you get B's 2's complement using XOR gates and the carry-in?

Designs

Circuit	Outputs
half adder	$S = A \oplus B$, $C = AB$
half subtractor (A − B)	$D = A \oplus B$, $B_o = A'B$
full adder	$S = A \oplus B \oplus C_{in}$, $C_{out} = AB + BC_{in} + AC_{in}$ $(= AB + C_{in}(A \oplus B))$
full subtractor	$D = A \oplus B \oplus B_i$, $B_o = A'B + (A \oplus B)'B_i$

A B C_{in}	000	001	010	011	100	101	110	111
full adder S C_{out}	0 0	1 0	1 0	0 1	1 0	0 1	0 1	1 1
full subtractor D B_o (third input = B_i)	0 0	1 1	1 1	0 1	1 0	0 0	0 0	1 1

Wiring

Circuit	Wires (7486/7408/7432: gate 1 = pins 1, 2 → 3; gate 2 = 4, 5 → 6; gate 3 = 9, 10 → 8; gate 4 = 12, 13 → 11)
half adder	A → 7486-1 and 7408-1; B → 7486-2 and 7408-2. **S = 7486-3, C = 7408-3**
half subtractor	D = 7486 (A → 1, B → 2 → 3). A → 7404-1 (pin 2 = A'); 7404-2 → 7408-1, B → 7408-2; **B_o = 7408-3**
full adder: sum	7486 gate 1: A, B → 3; gate 2: pin 3 → 4, C_{in} → 5; **S = 7486-6**
full adder: carry (Fig 2.3)	7408: A, B → 3 (AB); B → 4, C_{in} → 5 → 6 (BC_{in}); A → 9, C_{in} → 10 → 8 (AC_{in}). 7432: 7408-3 → 1, 7408-6 → 2 → 3; 7432-3 → 4, 7408-8 → 5; **C_{out} = 7432-6**
full subtractor: borrow	7404: A → 1 → 2 (A'); 7486-3 (A ⊕ B) → 3 → 4. 7408: 7404-2 → 1, B → 2 → 3 (A'B); 7404-4 → 4, B_i → 5 → 6. 7432: 3 → 1, 6 → 2; **B_o = 7432-3**

The 7483: 4-bit adder (note the odd power pins)



VCC is pin 5 and GND is pin 12, not 16/8. Wiring power to 16 and 8 is the classic mistake with this chip.

Signal	A3 A2 A1 A0	B3 B2 B1 B0	C_{in}	S3 S2 S1 S0	C_{out}
pin	1, 3, 8, 10	16, 4, 7, 11	13	15, 2, 6, 9	14

4-bit adder / 2's-complement subtractor (Fig 2.6)

- Feed each B bit through an XOR whose other input is the control **M** (7486), and also send **M to C_{in}** (pin 13).
- **M = 0:** B passes unchanged, C_{in} = 0, so the 7483 computes **A + B**.
- **M = 1:** each B bit is inverted (1's complement) and C_{in} = 1 adds the 1, giving 2's complement. So the 7483 computes **A − B**.

Wire	
b0 → 7486-1, M → 2 → pin 3 → 7483-11	b1 → 7486-4, M → 5 → pin 6 → 7483-7
b2 → 7486-9, M → 10 → pin 8 → 7483-4	b3 → 7486-12, M → 13 → pin 11 → 7483-16
M → 7483-13 (C_{in})	A3..A0 → pins 1, 3, 8, 10

Try	Inputs	S3..S0	C_{out}	Meaning
7 + 5	M = 0, A = 0111, B = 0101	1100	0	12
7 − 5	M = 1, A = 0111, B = 0101	**0010**	**1**	+2. C_{out} = 1 means no borrow, result positive.
5 − 7	M = 1, A = 0101, B = 0111	**1110**	**0**	C_{out} = 0 means negative: 1110 is −2 in 2's complement

BCD adder (Fig 2.7)

- Add the two BCD digits in 7483 #1.
- If the sum is more than 9, or produced a carry, add 6 (0110) in 7483 #2.

The correction signal is:

$$K = C_4 + S_3S_2 + S_3S_1$$

Make S3S2 and S3S1 with two 7408 gates, then OR all three with two 7432 gates. Feed **0 K K 0** into B3 B2 B1 B0 of 7483 #2, with the first sum on its A inputs and C_{in} = 0. **K is the decimal carry.**

Example: 7 + 5. 7483 #1 gives 1100 (12 > 9), so K = 1. Then 1100 + 0110 = (1)0010. Output: carry **1**, digit **0010**, which reads as 12 ✓.

WRITE IN YOUR RECORD

Result: the truth tables of the half/full adder and half/full subtractor were verified. The 7483 added two 4-bit numbers; with the XOR/M arrangement it subtracted in 2's complement (7 − 5 = 0010, C_{out} = 1; 5 − 7 = 1110, C_{out} = 0). The BCD adder gave 7 + 5 = 1 0010.

Theory for the record: a half adder adds two bits (S = A ⊕ B, C = AB). A full adder adds three, including the carry-in (S = A ⊕ B ⊕ C_{in}, C_{out} = AB + BC_{in} + AC_{in}), and can be built from two half adders and an OR. Subtractors produce a difference and a borrow (half: D = A ⊕ B, B_o = A'B). The 7483 is a 4-bit binary adder with fast carry. Subtraction uses A − B = A + B' + 1, made by XOR-ing B with a control M and feeding M to C_{in}. For BCD addition, 6 is added when the binary sum exceeds 9 or produces a carry.

CHECK 1

(a) In the subtractor, why does C_{out} = 1 mean "no borrow"? (b) BCD 8 + 9: give 7483 #1's output, K, and the final output. (c) A student powers the 7483 on pins 16 and 8. What happens?

Answers

Opening question. XOR each B bit with M (M = 1 inverts it) and feed M into C_{in} (+1). Together that adds the 2's complement of B, so the adder subtracts. After a subtraction, C_{out} = 1 means A ≥ B (positive result); C_{out} = 0 means the result is negative, in 2's-complement form.

Check 1. (a) A + (2^4 − B) = 2^4 + (A − B). When A ≥ B this overflows 4 bits, so carry = 1. When A < B it doesn't. (b) 8 + 9 = 17: #1 gives C_4 = 1, S = 0001, so K = 1. 0001 + 0110 = **0111**, carry 1, i.e. **1 0111** = 17 ✓. (c) Pin 16 is B3 and pin 8 is A1, so the chip gets no power. If the real VCC/GND pins (5, 12) are left floating while inputs are driven, it misbehaves and can be damaged. The outputs are garbage.

Exp 3 · f(W,X,Y,Z) = Σm(1,3,5,7,8,12,14) with basic gates, NAND only, NOR only

Manual p. 10 · ICs 7404, 7408, 7411, 7432 (basic); 7400 + 7410 (NAND); 7402 + 7427 (NOR) · all four forms checked over all 16 inputs

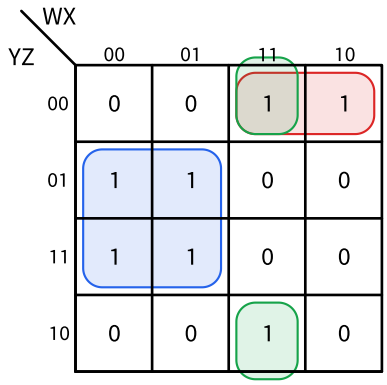
MANUAL EXP 3, AIM

Realise f(W, X, Y, Z) = Σm(1, 3, 5, 7, 8, 12, 14) using (i) basic gates, (ii) only NAND gates, (iii) only NOR gates.

GUESS FIRST

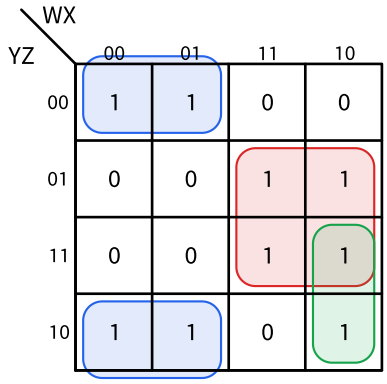
Which form do you minimise for NAND-only, and which for NOR-only: SOP or POS?

Minimise



K-map of f (the 1s). Blue: W'Z (m1, 3, 5, 7); red: WY'Z' (m8, 12); green: WXZ' (m12, 14).

f = W'Z + WY'Z' + WXZ' (SOP, for basic gates and for NAND–NAND)

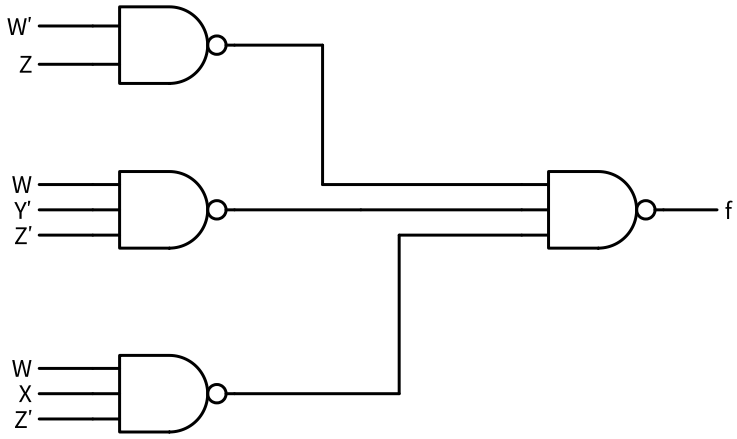


K-map of f' (the 0s). Blue: W'Z'; red: WZ; green: WX'Y.

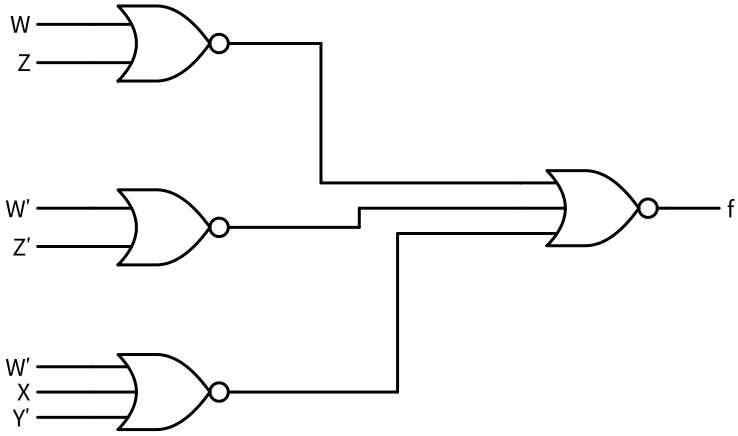
f' = W'Z' + WZ + WX'Y ⇒ f = (W + Z)(W' + Z')(W' + X + Y') (POS, for NOR–NOR)

Rule:

- SOP becomes **NAND–NAND**: replace every gate with a NAND, and keep the inverters.
- POS becomes **NOR–NOR**: replace every gate with a NOR.



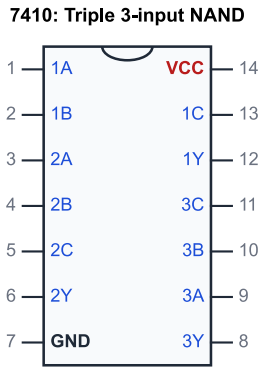
NAND–NAND realisation of the SOP. The inverters W', Y', Z' are NANDs with their inputs tied.



NOR–NOR realisation of the POS.

Wiring

Form	Wires
basic gates	7404: W → 1 → 2 (W'); Y → 3 → 4 (Y'); Z → 5 → 6 (Z'). 7408: 7404-2 → 1, Z → 2 → **3** (W'Z). 7411: W → 1, 7404-4 → 2, 7404-6 → 13 → **12** (WY'Z'); W → 3, X → 4, 7404-6 → 5 → **6** (WXZ'). 7432: 7408-3 → 1, 7411-12 → 2 → 3; 7432-3 → 4, 7411-6 → 5; **f = 7432-6**
NAND only	7400: W → 1, 2 → 3 (W'); Y → 4, 5 → 6 (Y'); Z → 9, 10 → 8 (Z'); 7400-3 → 12, Z → 13 → **11**. 7410: W → 1, 7400-6 → 2, 7400-8 → 13 → **12**; W → 3, X → 4, 7400-8 → 5 → **6**; 7400-11 → 9, 7410-12 → 10, 7410-6 → 11; **f = 7410-8**
NOR only	7402: W → 2, 3 → 1 (W'); Y → 5, 6 → 4 (Y'); Z → 8, 9 → 10 (Z'); W → 11, Z → 12 → **13** (W + Z)'. 7427: 7402-1 → 1, 7402-10 → 2, **GND** → **13** → **12** (W' + Z')'; 7402-1 → 3, X → 4, 7402-4 → 5 → **6**; 7402-13 → 9, 7427-12 → 10, 7427-6 → 11; **f = 7427-8**



7410 (3-input NAND); the 7411 (AND) and 7427 (NOR) share this layout.

Truth table to verify

WXYZ	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
f	0	1	0	1	0	1	0	1	1	0	0	0	1	0	1	0

TRAP: THE SPARE NOR INPUT

The 7427 has only 3-input NORs, and the term (W' + Z')' needs 2 inputs. Tie the third input to **GND**: 0 does not change an OR. Left floating, it reads as 1 and the gate's output sticks at 0.

WRITE IN YOUR RECORD

Result: f = Σm(1, 3, 5, 7, 8, 12, 14) was minimised to f = W'Z + WY'Z' + WXZ' (SOP) and f = (W + Z)(W' + Z')(W' + X + Y') (POS). It was realised with basic gates, with NAND gates only and with NOR gates only, and all three circuits matched the truth table.

Theory for the record: a Boolean function is minimised with a K-map by grouping adjacent 1s (giving SOP) or 0s (giving POS) in powers of two. A two-level SOP (AND–OR) converts directly to NAND–NAND, and a POS (OR–AND) to NOR–NOR, by De Morgan's theorem. Inverters are made from NAND/NOR gates with their inputs tied. Universal-gate realisations need only one IC type.

CHECK 1

(a) Prove WY'Z' + WXZ' = WZ'(Y' + X) by K-map or algebra, and say why the pack keeps it as two terms. (b) In the NAND circuit, what is the output of the 7400 gate on pins 12–13 when W = 0, Z = 1? (c) Count the ICs in each version.

Answers

Opening question. NAND-only: **SOP** (AND–OR → NAND–NAND). NOR-only: **POS** (OR–AND → NOR–NOR).

Check 1. (a) Factor out WZ'. Keeping two terms gives a pure two-level circuit, which converts gate-for-gate to NAND–NAND. The factored form is three levels. (b) Inputs are W' = 1 and Z = 1, so NAND = **0**. The final NAND then outputs f = 1 ✓ (m1, m3, … all have W = 0, Z = 1). (c) Basic: 4 ICs (7404, 7408, 7411, 7432). NAND: 2 (7400, 7410). NOR: 2 (7402, 7427).

Exp 4 · Code converters: binary → Gray, BCD → Excess-3

Manual pp. 11–12 · ICs 7486, 7432, 7408, 7404 · both converters checked over every valid input

MANUAL EXP 4, AIM

Design and implement (A) a 4-bit binary to Gray code converter, (B) a BCD to Excess-3 converter. Viva form: "Why is Gray code used in position encoders, and why is Excess-3 self-complementing?"

GUESS FIRST

How many bits change between consecutive Gray codes? What is 9's complement of 2 in Excess-3?

A · Binary → Gray

$$G_3 = B_3, \qquad G_2 = B_3 \oplus B_2, \qquad G_1 = B_2 \oplus B_1, \qquad G_0 = B_1 \oplus B_0$$

Wiring (manual's pins):

- B3 → output G3 directly.
- 7486 gate 1: B3 → 1, B2 → 2 → **3 = G2**.
- 7486 gate 2: B2 → 4, B1 → 5 → **6 = G1**.
- 7486 gate 3: B1 → 9, B0 → 10 → **8 = G0**.

B	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
G	0000	0001	0011	0010	0110	0111	0101	0100	1100	1101	1111	1110	1010	1011	1001	1000

Gray → binary (exercise): B3 = G3, B2 = B3 ⊕ G2, B1 = B2 ⊕ G1, B0 = B1 ⊕ G0. It is a chain: each XOR uses the *previous output*, not the previous input.

B · BCD → Excess-3 (XS-3 = BCD + 3)

BCD ABCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
XS-3 WXYZ	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

The K-maps use 1010–1111 as don't-cares (they are not BCD). The manual's expressions, and a factored form that saves gates:

$$W = A + BC + BD = A + B(C + D) \qquad X = B'C + B'D + BC'D' = B \oplus (C + D)$$

$$Y = CD + C'D' = (C \oplus D)' \qquad Z = D'$$

Wiring (factored form):

Gate	Wires
7432 gate 1	C → 1, D → 2 → 3 (C + D)
7408 gate 1	B → 1, 7432-3 → 2 → 3 (B(C + D))
7432 gate 2	A → 4, 7408-3 → 5 → **6 = W**
7486 gate 1	B → 1, 7432-3 → 2 → **3 = X**
7486 gate 2	C → 4, D → 5 → 6; then 7404: 7486-6 → 1 → **2 = Y**
7404	D → 3 → **4 = Z**

TRAP: INPUTS ABOVE 1001

Set 1010–1111 and the outputs look like nonsense. Those inputs aren't BCD, and the design treated them as don't-cares. Test only 0000–1001.

WRITE IN YOUR RECORD

Result: the binary-to-Gray converter (G3 = B3, Gi = Bi+1 ⊕ Bi) and the BCD-to-Excess-3 converter (W = A + BC + BD, X = B'C + B'D + BC'D', Y = CD + C'D', Z = D') were implemented and their truth tables verified.

Theory for the record: a code converter changes one binary code into another. In Gray code, successive values differ in one bit only. That avoids false intermediate readings in shaft encoders, and it gives the order of K-map labels. Binary → Gray: $G_n = B_n$, $G_i = B_{i+1} \oplus B_i$. Excess-3 is BCD + 3. It is self-complementing: inverting all bits gives the 9's complement, which simplifies decimal subtraction. Each output bit is found from a K-map, with the six unused BCD codes as don't-cares.

CHECK 1

(a) Convert Gray 1011 to binary. (b) Show XS-3 is self-complementing using 2 and 7. (c) Why can X = B'C + B'D + BC'D' be written B ⊕ (C + D)?

Answers

Opening question. One bit changes between consecutive Gray codes, so a sensor caught mid-transition can be off by at most one step. 2 in XS-3 is 0101; inverting gives 1010 = XS-3 of 7 = 9 − 2. So the 9's complement is just the bitwise inverse.

Check 1. (a) B3 = 1, B2 = 1 ⊕ 0 = 1, B1 = 1 ⊕ 1 = 0, B0 = 0 ⊕ 1 = 1, giving **1101**. (b) XS-3(2) = 0101 and XS-3(7) = 1010: bitwise inverses, and 2 + 7 = 9. (c) B'C + B'D = B'(C + D), and BC'D' = B(C + D)'. B'P + BP' with P = C + D is exactly B ⊕ P.

Exp 5 · Magnitude comparator (1-bit and 7485) and odd-parity generator/checker (gates and 74180)

Manual pp. 13–18 · ICs 7404, 7408, 7486, 7485, 74180

3 NAND/NOR design → 4 Code converters → **5 Comparator & parity** → 6–8 MUX, DEMUX, decoder

MANUAL EXP 5, AIM

Design and study 1-bit and 4-bit (7485) magnitude comparators, and a 3-bit odd-parity generator/checker, then the 74180. Viva form: "A lone 7485 lights both A>B and A<B when the inputs are equal. What did you forget?"

GUESS FIRST

The 7485 has three *cascade inputs*. What must they be set to when only one chip is used?

1-bit comparator

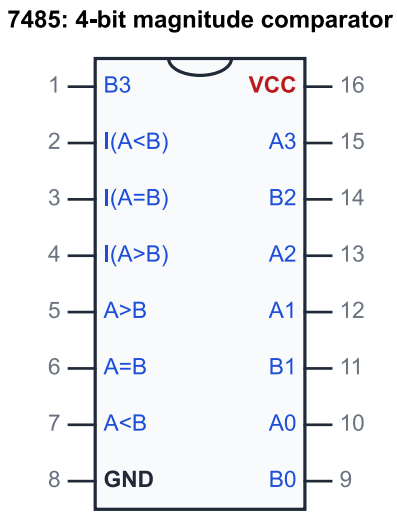
$$(A > B) = AB' \qquad (A < B) = A'B \qquad (A = B) = (A \oplus B)'$$

Wiring:

- 7404: A → 1 → 2 (A'); B → 3 → 4 (B').
- 7408: A → 1, 7404-4 → 2 → **3 = (A > B)**; 7404-2 → 4, B → 5 → **6 = (A < B)**.
- 7486: A → 1, B → 2 → 3; 7404: 7486-3 → 5 → **6 = (A = B)**.

Only one output is HIGH at a time.

4-bit comparator, 7485



7485. The cascade inputs I(A>B), I(A=B), I(A

Signal	A3 A2 A1 A0	B3 B2 B1 B0	I(A>B)	I(A=B)	I(A<B)	A>B, A=B, A<B out
pin	15, 13, 12, 10	1, 14, 11, 9	4 → **GND**	3 → **+5 V**	2 → **GND**	5, 6, 7

The chip compares from the MSB down. The first unequal bit decides the result. Only when all four bit pairs are equal does it pass the **cascade inputs** through. So a single chip needs I(A=B) = 1 (and the others 0) for "equal" to come out right.

8-bit (exercise):

- The **low** nibble's chip has cascade inputs 0, 1, 0.
- Its three outputs (pins 5, 6, 7) go into the **high** nibble chip's cascade inputs (pins 4, 3, 2).
- The final answer comes from the high chip.

Odd parity generator and checker

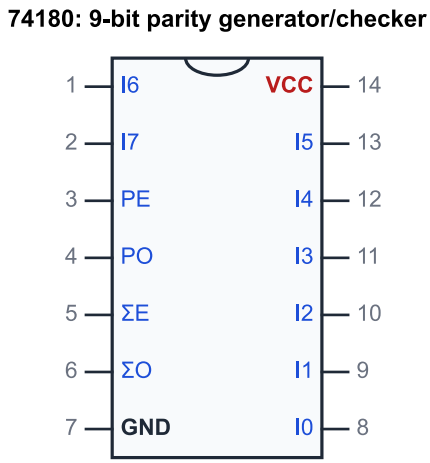
Odd parity makes the total number of 1s (data + parity) **odd**.

- **Generator:** P = (X ⊕ Y ⊕ Z)'. Two XORs (7486) and an inverter (7404). The manual's Fig 5.4 feeds Y, Z into gate 1 and X with gate 1's output into gate 2.
- **Checker:** C = (X ⊕ Y ⊕ Z ⊕ P)'. Three XORs and an inverter. **C = 1 flags an error** (an even count arrived).

Manual Fig 5.5 puts inverters after the first two XORs. Those two inversions cancel inside the last XOR, so the result is the same.

XYZ	000	001	010	011	100	101	110	111
P (odd)	1	0	0	1	0	1	1	0

74180: 9-bit parity generator/checker



74180. Data I0–I7; enables PE (pin 3) and PO (pin 4);
outputs ΣE (pin 5) and ΣO (pin 6).

Count of 1s in I0–I7	PE	PO	ΣE	ΣO
even	1	0	1	0
odd	1	0	0	1
even	0	1	0	1
odd	0	1	1	0
any	1	1	0	0
any	0	0	1	1

The manual's observation table, filled (checked):

I7...I0	PE PO	ΣE ΣO
1111 0000	1 0	**1 0**
1111 0001	1 0	**0 1**
1010 1100	0 1	**0 1**
1010 1101	0 1	**1 0**
0000 0000	1 0	**1 0**
1111 1111	0 1	**0 1**

WRITE IN YOUR RECORD

Result: the 1-bit comparator and the 7485 4-bit comparator gave exactly one HIGH output per input pair. The 3-bit odd-parity generator/checker was verified (checker output 1 = error). The 74180 outputs matched the observation table.

Theory for the record: a magnitude comparator decides A > B, A = B or A < B. For one bit: AB', A'B and (A ⊕ B)'. The 7485 compares two 4-bit words from the MSB down. Its cascade inputs let chips be chained for longer words, and must be set to (0, 1, 0) for a single chip. A parity bit makes the number of 1s even or odd so that single-bit errors can be detected. Odd-parity generator: P = (X ⊕ Y ⊕ Z)'. The checker XORs all received bits. The 74180 generates/checks parity on 8 bits, with PE/PO selecting even/odd.

CHECK 1

(a) A = 1010, B = 1010 on a lone 7485 with I(A=B) left floating. What do you expect, and why is "floating" risky here? (b) Odd parity, data 101: give P, and the checker output if the received word is 1011. (c) How do you make a 9-bit odd-parity checker from one 74180 and an inverter?

Answers

Opening question. The cascade inputs: I(A=B) (pin 3) must be **HIGH**, and I(A>B), I(A<B) (pins 4, 2) **LOW**.

Check 1. (a) A floating TTL input usually acts HIGH, so it may happen to work. That relies on undefined behaviour, so tie pin 3 to +5 V. The datasheet's function table shows what wrong cascade settings do when A = B: all three at 0 gives A>B = A**B) = I(A<B) = 1 with I(A=B) = 0 gives all three outputs 0. (b) P = (1 ⊕ 0 ⊕ 1)' = 1. The word 1011 has three 1s (odd) = valid, so the checker output is 0 (no error). (c) Feed 8 bits to I0–I7. Feed the 9th bit to PO, and its inverse (via the inverter) to PE. Then ΣO/ΣE give the odd-parity check of all 9 bits.**

Exps 6–8 · Multiplexers, demultiplexers, decoders

Manual pp. 19–24 and 2–3 (second numbering) · ICs 7410, 7420, 7411, 7404, 7408, 74151, 74153, 74157, 74138 · MUX data inputs checked over all 16 minterms

5 Comparator & parity → **6 MUX** → 7 DEMUX → 8 Decoder → 9–10 Flip-flops

MANUAL EXP 6 (II)

Implement f = Σm(0, 3, 5, 7, 8, 10, 14) using (i) an 8:1 MUX, (ii) a 4:1 MUX and additional gates. Viva form: "With 4 variables and an 8:1 MUX, what goes on the data inputs?"

GUESS FIRST

Three variables go to the select lines. What can each data input be, in terms of the fourth variable?

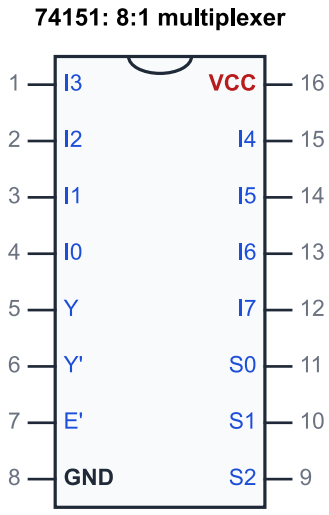
Exp 6 · Multiplexer

4:1 MUX: Y = S1'S0'D0 + S1'S0D1 + S1S0'D2 + S1S0D3.

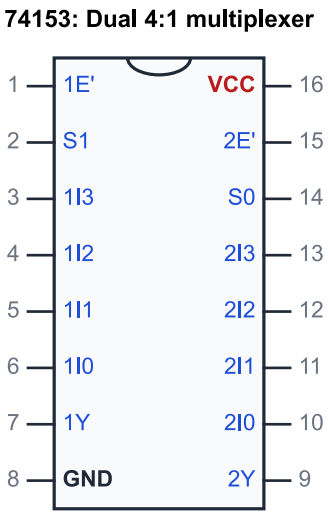
NAND-only version (Fig 6.2):

- Two 3-input NANDs with tied inputs make S1' and S0'.
- Four 3-input NANDs (D_i, select terms) feed one 4-input NAND (7420).
- That is two 7410s and one 7420.

Gate	Wires
7410 #1	S1 → 1, 2, 13 → **12 = S1'**; S0 → 3, 4, 5 → **6 = S0'**; D0 → 9, S1' → 10, S0' → 11 → 8
7410 #2	D1 → 1, S1' → 2, S0 → 13 → 12; D2 → 3, S1 → 4, S0' → 5 → 6; D3 → 9, S1 → 10, S0 → 11 → 8
7420	#1-8 → 1, #2-12 → 2, #2-6 → 4, #2-8 → 5 → **6 = Y** (pin 3 is NC)



74151 (8:1). E' (pin 7) must be LOW to enable. Y (pin 5), Y' (pin 6).



74153 (dual 4:1). S0 = pin 14, S1 = pin 2 are shared; each half has its own E'.

f = Σm(0,3,5,7,8,10,14) on the 74151 (select = A B C, data in terms of D):

Select ABC	000	001	010	011	100	101	110	111
minterms	0, 1	2, 3	4, 5	6, 7	8, 9	10, 11	12, 13	14, 15
f values	1, 0	0, 1	0, 1	0, 1	1, 0	1, 0	0, 0	1, 0
data input	**I0 = D'**	**I1 = D**	**I2 = D**	**I3 = D**	**I4 = D'**	**I5 = D'**	**I6 = 0**	**I7 = D'**
74151 pin	4	3	2	1	15	14	13	12

Wiring:

- A → pin 9 (S2), B → 10 (S1), C → 11 (S0).
- D → pins 3, 2, 1. D' (from 7404: D → 1 → 2) → pins 4, 15, 14, 12.
- Pin 13 → GND. E' (7) → GND.
- **f = pin 5.**

On the 74153 (select = A B, data in terms of C, D): I0 = (C ⊕ D)', I1 = D, I2 = D', I3 = CD'.

- S1 (pin 2) ← A, S0 (14) ← B.
- 1I0 (6) ← 7486(C, D) through a 7404 inverter.
- 1I1 (5) ← D; 1I2 (4) ← D'; 1I3 (3) ← 7408(C, D').
- 1E' (1) → GND. **f = 1Y (pin 7).**



74157 (quad 2:1). One select S (pin 1), common enable E' (pin 15). Test: S = 0 → each Y = A input; S = 1 → each Y = B input.

Exp 7 · Demultiplexer

1:4 demux with basic gates (Fig 7.1): Y0 = D S1'S0', Y1 = D S1'S0, Y2 = D S1S0', Y3 = D S1S0.

- 7404: S1 → 1 → 2 (S1'); S0 → 3 → 4 (S0').
- 7411 #1: D → 1, S1' → 2, S0' → 13 → **12 = Y0**; D → 3, S1' → 4, S0 → 5 → **6 = Y1**; D → 9, S1 → 10, S0' → 11 → **8 = Y2**.
- 7411 #2: D → 1, S1 → 2, S0 → 13 → **12 = Y3**.



74138. Address A0–A2 (pins 1–3), enables E1', E2' (active LOW) and E3 (active HIGH), outputs O0'–O7' (active LOW).

1:8 demux on the 74138 (the manual's parts list wrongly says 74151/74153):

- Select → A0 (pin 1, LSB), A1 (2), A2 (3).
- **Data D → E1' (pin 4)**; E2' (5) → GND; E3 (6) → +5 V.
- When D = 0 the chip is enabled, so the selected output goes LOW = D. When D = 1 it is disabled, so all outputs are HIGH, and the selected one again equals D.
- So **the selected output follows D, and every other output stays 1.** Outputs O0'…O7' are pins 15, 14, 13, 12, 11, 10, 9, 7.

Exp 8 · Decoder

2:4 decoder with basic gates (Fig 8.1): D0 = A'B', D1 = A'B, D2 = AB', D3 = AB.

- 7404: A → 1 → 2, B → 3 → 4.
- 7408: 7404-2, 7404-4 → 1, 2 → **3 = D0**; 7404-2, B → 4, 5 → **6 = D1**; A, 7404-4 → 9, 10 → **8 = D2**; A, B → 12, 13 → **11 = D3**.

2:4 decoder with enable, NAND only (Fig 8.2): outputs are **active-LOW**: $D_i' = (E \cdot \text{minterm}_i)'$.

- 7410 #1: A → 1, 2, 13 → 12 (A'); B → 3, 4, 5 → 6 (B'); E → 9, A' → 10, B' → 11 → **8 = D0'**.
- 7410 #2: E, A', B → 1, 2, 13 → **12 = D1'**; E, A, B' → 3, 4, 5 → **6 = D2'**; E, A, B → 9, 10, 11 → **8 = D3'**.

E A B	0 x x	1 0 0	1 0 1	1 1 0	1 1 1
D0' D1' D2' D3'	1 1 1 1	**0** 1 1 1	1 **0** 1 1	1 1 **0** 1	1 1 1 **0**

TRAP: ACTIVE-LOW OUTPUTS

On the trainer, the selected output's LED goes **out** while the others stay lit, for both the NAND decoder and the 74138. That's correct, not a fault.

WRITE IN YOUR RECORD

Result: the 4:1 MUX (NAND gates) and the 74151/74153/74157 function tables were verified. f = Σm(0, 3, 5, 7, 8, 10, 14) was realised on the 74151 (I0 = D', I1 = I2 = I3 = D, I4 = I5 = I7 = D', I6 = 0) and on the 74153 with gates. The 1:4 DEMUX (gates), the 1:8 DEMUX (74138, data on E1'), and the 2:4 decoders (basic gates; NAND with enable, active-low) were verified.

Theory for the record: a multiplexer selects one of 2^n data inputs using n select lines: $Y = \Sigma\, m_i D_i$. Any function of n + 1 variables can be built on a 2^n:1 MUX by putting n variables on the selects and 0, 1, x or x' on each data input. A demultiplexer routes one input to one of 2^n outputs. A decoder activates exactly one of 2^n outputs for an n-bit input. A decoder with an enable becomes a demultiplexer when the data is applied to the enable.

CHECK 1

(a) Full adder on one 8:1 MUX (select A, B, C_{in}): list the data inputs for S and for C_{out}. (b) The 74151's E' is left floating. What do you see? (c) Why does a decoder plus an OR gate realise any function?

Answers

Opening question. Each data input is **0, 1, D or D'**. Read each pair of minterms that share the same select value (see the table).

Check 1. (a) S: I1, I2, I4, I7 = 1, others 0. C_{out}: I3, I5, I6, I7 = 1, others 0 (two 74151s, or one 74153 per output with S = C_{in}, C_{in}', C_{in}', C_{in} and C_{out} = 0, C_{in}, C_{in}, 1). (b) Floating acts HIGH, which disables the chip: Y stays 0 and Y' stays 1 whatever the inputs. (c) The decoder produces every minterm. OR-ing the minterms in Σm gives the function. (With active-low outputs, use a NAND instead of the OR.)

Exps 9–10 · Flip-flops from NAND gates; 7474 and 7473; flip-flop conversion

Manual pp. 4–8 (second numbering) · ICs 7400, 7410, 7474, 7473 · use the trainer's debounced pulser as the clock

MANUAL EXPS 9–10

Study SR, JK and T flip-flops built from NAND gates, the 7474 (positive-edge D, preset and clear) and the 7473 (negative-edge JK, clear), and conversions SR → D, D → SR. Viva form: "Your NAND JK flip-flop with J = K = 1 ends in a random state. Why, and how do the ICs avoid it?"

GUESS FIRST

The gate JK is *level*-triggered: it reacts for as long as CLK = 1. What happens if Q flips while CLK is still 1?

Exp 9 · From gates

Clocked SR (Fig 9.1), one 7400:

- G1 = NAND(S, CLK): 1, 2 → 3.
- G2 = NAND(R, CLK): 4, 5 → 6.
- G3 = NAND(G1, Q'): 3 → 9, 11 → 10 → **8 = Q**.
- G4 = NAND(G2, Q): 6 → 12, 8 → 13 → **11 = Q'**.

CLK	S R	Q_{n+1}
0	x x	Q_n (hold)
1	0 0	Q_n
1	0 1	0
1	1 0	1
1	1 1	**invalid** (Q = Q' = 1, then unpredictable when CLK falls)

JK (Fig 9.2): feed Q' back into the J gate and Q into the K gate. That makes S = R = 1 impossible, and J = K = 1 toggles instead.

- 7410: J → 1, CLK → 2, Q' → 13 → 12; K → 3, CLK → 4, Q → 5 → 6.
- 7400 latch: 7410-12 → 1, Q' → 2 → **3 = Q**; 7410-6 → 4, Q → 5 → **6 = Q'**.

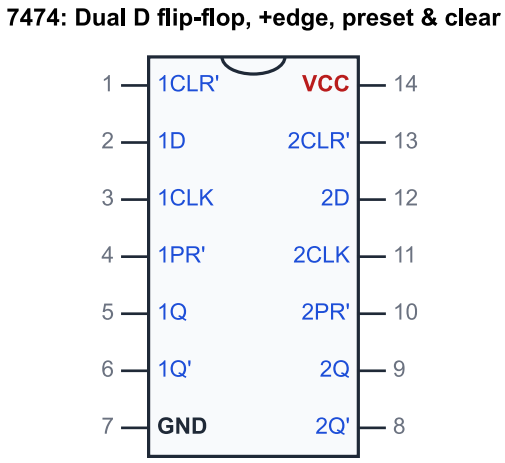
CLK pulse	J K	Q_{n+1}
↑ rising	0 0	Q_n
↑ rising	0 1	0
↑ rising	1 0	1
↑ rising	1 1	Q_n' (toggle)

RACE-AROUND

This JK is level-triggered. With J = K = 1, it toggles, the new Q feeds back, and it toggles again, as many times as fit inside one CLK-high time. The final state on the trainer is random. Edge-triggered ICs (7473, 7474) and master-slave designs act only at a clock **edge**, so each pulse gives exactly one toggle.

T flip-flop (Fig 9.3): tie J = K = T. T = 0 holds; T = 1 toggles on every clock.

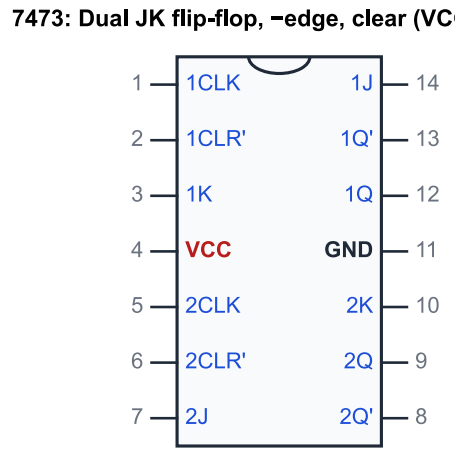
Exp 10 · Flip-flop ICs



7474: two positive-edge D flip-flops with active-LOW preset and clear.

7474 (FF 1): CLR' 1, D 2, CLK 3, PR' 4, Q 5, Q' 6.

PR'	CLR'	CLK	D	Q Q'
0	1	x	x	1 0 (preset, ignores clock)
1	0	x	x	0 1 (clear)
0	0	x	x	1 1 (not allowed)
1	1	↑ rising	1	1 0
1	1	↑ rising	0	0 1
1	1	0	x	Q_n Q_n'



7473: two negative-edge JK flip-flops with active-LOW clear. VCC = pin 4, GND = pin 11.

7473 (FF 1): CLK 1, CLR' 2, K 3, J 14, Q 12, Q' 13. **Table 10.2, filled:**

CLR'	CLK	J K	Q_{n+1}	Q_{n+1}'
0	x	x x	0	1
1	↓ falling	0 0	Q_n	Q_n'
1	↓ falling	1 0	1	0
1	↓ falling	0 1	0	1
1	↓ falling	1 1	Q_n'	Q_n (toggle)
1	1 (no edge)	x x	Q_n	Q_n'

Conversions:

Want	Using	Logic
D from SR (Fig 10.1)	S = D, R = D'	7404 inverts D for R
SR from D (Fig 10.2)	D = S + R'Q	7404 (R'), 7408 (R'·Q), 7432 (+S) into the 7474's D; Q fed back
JK from T (exercise)	T = JQ' + KQ	
T from D (exercise)	D = T ⊕ Q	one 7486
SR from JK (exercise)	J = S, K = R	S = R = 1 must still be avoided
SR from NOR (exercise)	cross-coupled NORs	active-HIGH inputs; S = R = 1 forbidden

WRITE IN YOUR RECORD

Result: the truth tables of the NAND-gate SR, JK and T flip-flops and of the 7474 (positive-edge D, PR'/CLR') and 7473 (negative-edge JK, CLR') were verified. SR → D (S = D, R = D') and D → SR (D = S + R'Q) conversions were verified.

Theory for the record: a flip-flop is a 1-bit memory element. The clocked SR flip-flop has an invalid state for S = R = 1. The JK removes it by feedback, giving toggle for J = K = 1. A level-triggered JK suffers race-around, which edge triggering or master-slave construction prevents. The D flip-flop copies D at the clock edge; the T flip-flop toggles when T = 1. Preset and clear are asynchronous, active-LOW inputs that override the clock. Conversion uses the excitation table of the available flip-flop to find its inputs in terms of the desired inputs and Q.

CHECK 1

(a) On the 7473, CLR' = 1, J = K = 1, Q = 0. Give Q after three clock pulses. (b) Why do you see no change on the 7474 while you hold the pulser down? (c) Derive D for a T flip-flop from the D excitation ($D = Q_{n+1}$).

Answers

Opening question. Race-around: with J = K = 1 and a long CLK-high, the gate JK toggles repeatedly through its own feedback, so the final state is random. ICs trigger on a clock **edge** (7474 ↑, 7473 ↓), so they change once per pulse.

Check 1. (a) It toggles each falling edge: 0 → 1 → 0 → **1**. (b) The 7474 acts only on the **rising** edge, which happens when you press. Holding the button is a steady level, so nothing more happens. (A 7473 acts on the release, the falling edge.) (c) $Q_{n+1} = Q_n$ when T = 0 and Q_n' when T = 1, so **D = T ⊕ Q**.

Exps 11–12 · Synchronous counter (7473), ring and Johnson counters, shift registers (7474)

Manual pp. 9–14 (second numbering) · ICs 7473 ×2, 7408, 7474 ×2, 7400 · every sequence below was simulated clock by clock

9–10 Flip-flops → **11 Counters** → 12 Shift registers → 13 FSM

MANUAL EXP 11, AIM

Design and test a 4-bit synchronous up counter using the 7473, and test ring and Johnson counters. Viva form: "Derive J and K for each flip-flop of the synchronous counter, and say why the ring counter needs a preset."

GUESS FIRST

In a binary count, when does bit QC toggle?

4-bit synchronous up counter

From the JK excitation table (0→0: 0x, 0→1: 1x, 1→0: x1, 1→1: x0), the manual's state table reduces to:

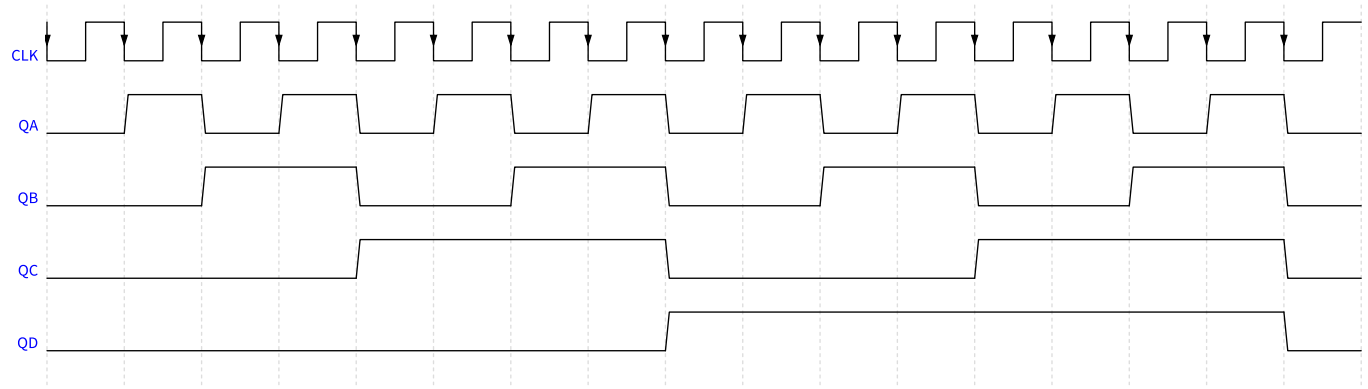
$$J_A = K_A = 1 \qquad J_B = K_B = Q_A \qquad J_C = K_C = Q_A Q_B \qquad J_D = K_D = Q_A Q_B Q_C$$

So each bit toggles exactly when all lower bits are 1. All clocks are common: that is what makes it **synchronous**.

Flip-flop	Pins (7473: FF1 = J 14, K 3, CLK 1, CLR' 2, Q 12; FF2 = J 7, K 10, CLK 5, CLR' 6, Q 9)
A = 7473 #1 FF1	J 14, K 3 → +5 V; **QA = 12**
B = 7473 #1 FF2	J 7, K 10 ← QA; **QB = 9**
C = 7473 #2 FF1	J 14, K 3 ← 7408-3 (QA → 1, QB → 2); **QC = 12**
D = 7473 #2 FF2	J 7, K 10 ← 7408-6 (7408-3 → 4, QC → 5); **QD = 9**
all four	CLK (pins 1, 5 on both ICs) ← clock; CLR' (pins 2, 6) ← clear switch; **VCC pin 4, GND pin 11**

Procedure:

1. Clear: CLR' = 0, then 1. The count shows DCBA = 0000.
2. Apply clock pulses. The 7473 changes on the **falling** edge.
3. The count goes 0000 → 0001 → … → 1111, and on the 16th pulse back to **0000**.



Expected waveforms: QA toggles every falling edge, QB every 2nd, QC every 4th, QD every 8th (divide-by-2, 4, 8, 16).

Ring and Johnson counters (two 7474s)

Name the four flip-flops Q3, Q2, Q1, Q0: IC1 FF1, IC1 FF2, IC2 FF1, IC2 FF2.

7474 pins: FF1 = CLR' 1, D 2, CLK 3, PR' 4, Q 5, Q' 6; FF2 = Q' 8, Q 9, PR' 10, CLK 11, D 12, CLR' 13.

	Ring counter	Johnson (twisted-ring) counter
shift chain	Q3 (IC1-5) → D (IC1-12); Q2 (IC1-9) → IC2-2; Q1 (IC2-5) → IC2-12	same
feedback	**Q0 (IC2-9) → D3 (IC1-2)**	**Q0' (IC2-8) → D3 (IC1-2)**
clear line goes to	**PR' of Q3 (IC1-4)** and CLR' of Q2, Q1, Q0 (IC1-13, IC2-1, IC2-13)	CLR' of all four (IC1-1, IC1-13, IC2-1, IC2-13)
tie HIGH	CLR' of Q3 (IC1-1), unused PR's	all PR's
start state	1000	0000
sequence	1000 → 0100 → 0010 → 0001 → 1000	0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000
states (n FFs)	n = 4	2n = 8

TRAP: A RING COUNTER CLEARED TO 0000

The manual clears all four flip-flops. A ring counter at 0000 just circulates zeros forever. One flip-flop must be **preset** to 1, so the clear line goes to Q3's PR' instead of its CLR'. The Johnson counter starts fine from 0000.

Exp 12 · Shift registers (7474 ×2)

SISO / SIPO (Fig 12.2):

- Same chain as the ring counter, without feedback. Serial in → D3 (IC1-2); serial out = Q0 (IC2-9).
- All CLR' go to the clear line; all PR' are tied HIGH.

Enter 1, 1, 0, 1 (one bit per clock):

After clock	serial in	Q3 Q2 Q1 Q0
1	1	1 0 0 0
2	1	1 1 0 0
3	0	0 1 1 0
4	1	**1 0 1 1** (parallel out = 1011, the word entered)
5	0	0 1 0 1
6	0	0 0 1 0
7	0	0 0 0 1

Serial out (Q0) after clocks 4, 5, 6, 7 = **1, 1, 0, 1**: the first bit in comes out first.

PISO / PIPO (Fig 12.3): parallel load through the **preset** inputs.

- Each PR' is driven by NAND(PE, data bit) on a 7400:
- PE → 1, D → 2 → 3 → IC1-4;
- PE → 4, C → 5 → 6 → IC1-10;
- PE → 9, B → 10 → 8 → IC2-4;
- PE → 12, A → 13 → 11 → IC2-10.
- Parallel outputs go through a 7408 (each Q AND output-enable).

Procedure:

1. Clear (CLR = 0, PE = 0), then set CLR = 1.
2. Set D C B A on the switches.
3. PE = 1 presets the 1-bits, so the register holds DCBA. Then set PE = 0.
4. **PIPO:** output enable = 1, read W X Y Z.
5. **PISO:** clock with serial in = 0. The bits leave at Q0 one per clock.

WRITE IN YOUR RECORD

Result: the 4-bit synchronous counter counted 0000–1111 and returned to 0000 on the 16th pulse. The ring counter (preset 1000) cycled through 4 states and the Johnson counter through 8. SISO/SIPO: 1011 entered serially, read in parallel after 4 clocks and serially after 4–7 clocks. PISO/PIPO: parallel data loaded through preset and read out in parallel and serially.

Theory for the record: in a synchronous counter all flip-flops share the clock, and logic on J/K decides which toggle: each bit toggles when all lower bits are 1 (J = K = product of the lower Q's). This avoids the cumulative delay of ripple counters. A ring counter circulates a single 1 (n states, must be preset). A Johnson counter feeds back Q' (2n states). Shift registers move data one position per clock: SISO, SIPO, PISO and PIPO, according to how data enters and leaves.

CHECK 1

(a) Why is $J_D = K_D = Q_A Q_B Q_C$? (b) How many flip-flops does a mod-10 Johnson counter need? (c) After loading 1011 into the SIPO register, how many clocks until the register reads 0000?

Answers

Opening question. QC toggles whenever QA = QB = 1, i.e. after every 4th count. So $J_C = K_C = Q_A Q_B$. The ring counter needs a preset because a cleared ring (0000) has no 1 to circulate.

Check 1. (a) In binary counting, a bit flips exactly when every lower bit is 1 (…0111 → …1000). So D toggles only when A, B and C are all 1. (b) 2n = 10, so **5**. (c) Four more clocks with serial in = 0 (it reads 0000 after clock 8).

Exp 13 · FSM: overlapping sequence detector (1010, and 101), and an asynchronous circuit

Manual pp. 15–18 (second numbering) · ICs 7473, 7404, 7408, 7411, 7432, 7486 (and 7474 for the 101 version) · both detectors checked on 300 random input streams each

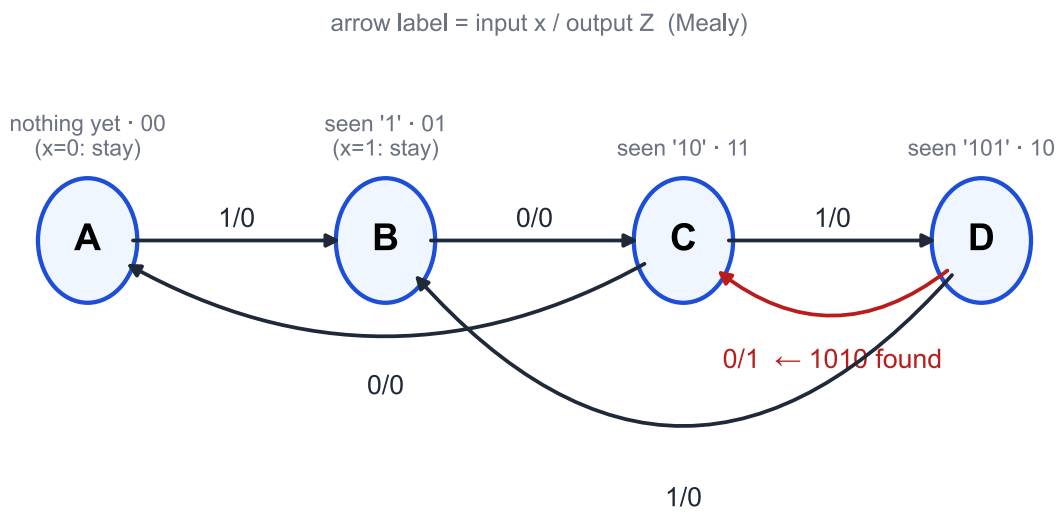
MANUAL EXP 13, AIM

Design a sequence detector that detects an overlapping sequence 1010 from an input sequence. (The list of experiments says **101**, so both are given.) Viva form: "Mealy or Moore? And what does *overlapping* change in the state diagram?"

GUESS FIRST

After 1010 is detected, which earlier state does the machine return to, and why not the start?

1010 detector (the manual's design)



Mealy state diagram. Each state remembers the longest useful suffix seen. After 1010, the last "10" is already the start of the next 1010, so the machine goes to C, not A.

State encoding (manual): A = 00, B = 01, C = 11, D = 10 (bits = flip-flop A, flip-flop B). With 7473 JK flip-flops, the K-maps give:

$J_A = BX'$ $K_A = BX' + B'X = B \oplus X$ $J_B = X + A$ $K_B = A$ $Z = AB'X'$

Signal	Wiring (7473: FF A = J 14, K 3, Q 12, Q' 13, CLK 1, CLR' 2; FF B = J 7, K 10, Q 9, Q' 8, CLK 5, CLR' 6)
X'	7404: X → 1 → 2
J_A	7408: QB (9) → 1, X' → 2 → 3 → 7473-14
K_A	7486: QB → 1, X → 2 → 3 → 7473-3
J_B	7432: X → 1, QA (12) → 2 → 3 → 7473-7
K_B	QA (12) → 7473-10
Z	7411: QA → 1, QB' (8) → 2, X' → 13 → **12 = Z**
clock, clear	pins 1, 5 ← pulser; pins 2, 6 ← clear; VCC 4, GND 11

Running it:

1. Clear to A.
2. For each bit: set X, **read Z first** (Mealy: Z depends on the present X), then pulse the clock.
3. Input 1, 0, 1, 0, 1, 0 gives Z = 0, 0, 0, **1**, 0, **1**. The second detection uses the overlap.

101 detector (the version in the experiment list)

States: S0 = nothing (00), S1 = seen "1" (01), S2 = seen "10" (10). With D flip-flops (7474), the logic shrinks to:

$$D_1 = Q_0X' \qquad D_0 = X \qquad Z = Q_1X$$

(JK form: $J_1 = Q_0X'$, $K_1 = 1$, $J_0 = X$, $K_0 = X'$.)

Wiring:

- 7474: FF1 holds Q1 (D 2, Q 5), FF2 holds Q0 (D 12, Q 9).
- X → 7474-12. 7408: Q0 (9) → 1, X' (7404) → 2 → 3 → 7474-2.
- Z: 7408: Q1 (5) → 4, X → 5 → **6**. CLK pins 3, 11; CLR' pins 1, 13 to clear; PR' 4, 10 HIGH.

Input 1, 0, 1, 0, 1 gives Z = 0, 0, **1**, 0, **1** (overlap: the final 1 of one "101" starts the next).

(ii) Fundamental-mode asynchronous circuit

Spec: Z follows X while Y = 1; Z holds while Y = 0. The manual reduces the six-state flow table to two states, giving

$$F^+ = XY + FY' \qquad Z = F$$

That is a gated D latch, with Y as the enable.

Wiring:

- 7408: X → 1, Y → 2 → 3 (XY).
- 7404: Y → 1 → 2 (Y').
- 7408: F → 4, Y' → 5 → 6 (FY').
- 7432: 7408-3 → 1, 7408-6 → 2 → **3 = F**, which is fed back to 7408-4. Z = F.

TRAP: THE LATCH CAN DROP OUT

With X = F = 1 and Y switching 1 → 0, the XY term turns off a moment before FY' turns on (the inverter delay). F can glitch to 0 and the latch loses its 1: a static-1 hazard. The cure is the consensus term **XF**: F^+ = XY + FY' + XF. It is the same function (checked), and hazard-free.

WRITE IN YOUR RECORD

Result: the overlapping 1010 Mealy detector (J_A = BX', K_A = B ⊕ X, J_B = X + A, K_B = A, Z = AB'X') produced Z = 1 at the 4th and 6th bits of 101010. The 101 detector ($D_1 = Q_0X'$, D_0 = X, $Z = Q_1X$) produced Z = 1 at the 3rd and 5th bits of 10101. The asynchronous circuit F^+ = XY + FY' transferred X to Z when Y = 1 and held it when Y = 0.

Theory for the record: a finite state machine has a finite set of states held in flip-flops. The next state depends on the present state and the inputs. In a Mealy machine the output depends on the state and the input; in a Moore machine on the state only. Design steps: state diagram → state table → state assignment → flip-flop excitation table → K-map minimisation → circuit. An overlapping detector returns, after a match, to the state for the longest suffix of the sequence that is also its prefix. A fundamental-mode asynchronous circuit has no clock: one input changes at a time and the circuit settles, and hazards must be removed with consensus terms.

CHECK 1

(a) Make the 1010 detector non-overlapping: where does the "0/1" arrow from D go? (b) Why must you read Z before pulsing the clock in a Mealy machine? (c) Moore version of 101: how many states?

Answers

Opening question. Mealy (Z = AB'X' uses the input X). After 1010 the machine goes to **C** ("10" seen), not A, because the trailing "10" can begin the next 1010. That is what *overlapping* means.

Check 1. (a) To **A** (start), so a match has to begin from scratch. (b) Z is a function of the present input. After the clock edge the state has moved on, and Z refers to the next bit. (c) **4**: S0, "1", "10", "101". The detection state is an extra state whose output is 1.

Exp 14 · TTL NAND and NOR gates (inside the chip)

In the experiment list, but the manual gives no pages for it · the circuit below is the standard 7400-type TTL NAND, simulated in LTspice (src/sim/ttl.py) with discrete transistors · theory: ECE2102 logic families

12 Shift registers → 13 FSM → **14 TTL**

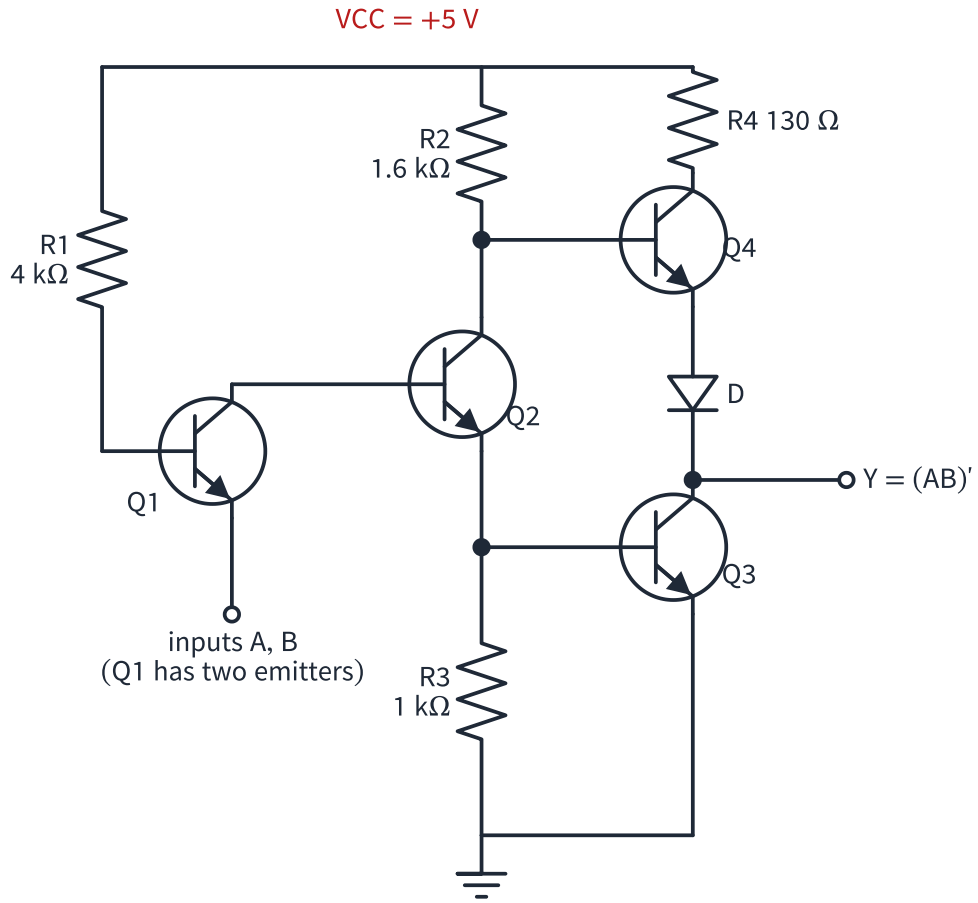
EXPERIMENT LIST, ITEM 14

Implement TTL logic for a NAND and a NOR gate. Viva form: "What does Q1's multi-emitter do, why is the output stage called a totem pole, and what is TTL's noise margin?"

GUESS FIRST

If any input is LOW, which way does Q1's base current go?

The TTL NAND



Standard TTL NAND: multi-emitter input (Q1), phase splitter (Q2), totem-pole output (Q4 on top, Q3 below). Build Q1 as two transistors with shared base and shared collector.

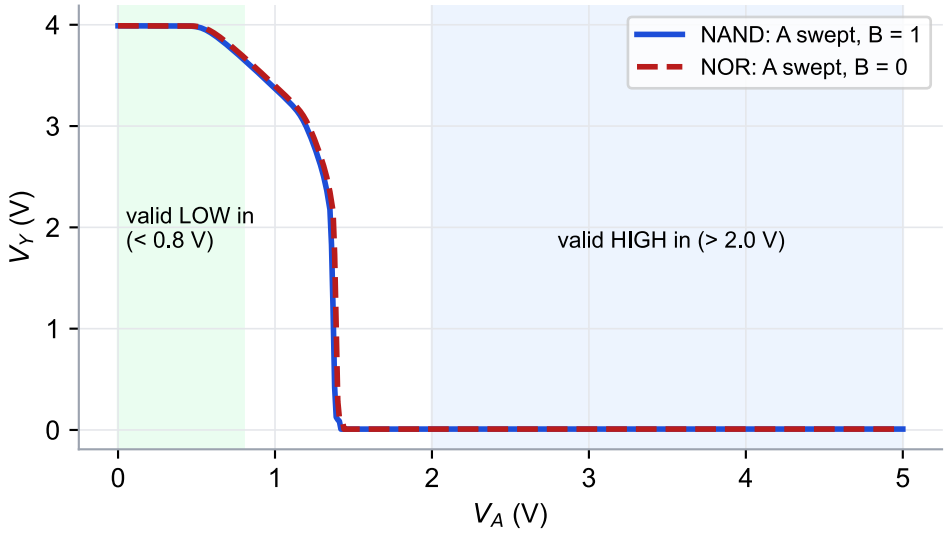
Inputs	What happens	Output
any input LOW	Q1's base current flows out through that emitter, so Q1 pulls Q2's base low. Q2 and Q3 are off; Q4 (via R2) and D conduct.	**HIGH** ≈ 5 − V_{BE4} − V_D ≈ 3.6 V when loaded (≈ 4 V lightly loaded)
all inputs HIGH	Q1's emitters are reverse-biased. R1's current flows through Q1's base–collector junction into Q2's base: Q2 and Q3 saturate. D keeps Q4 off.	**LOW** ≈ $V_{CE(sat)}$ ≈ 0.2 V

Totem pole: Q4 actively pulls the output up and Q3 actively pulls it down, stacked one above the other. That gives fast switching both ways. The 130 Ω resistor and the diode stop both transistors conducting hard at once.

TTL NOR: each input gets its own Q1 + Q2 pair. The two Q2s sit **in parallel** (shared collector and emitter), so **any** HIGH input turns Q3 on and pulls the output LOW. The totem pole is the same.

Results (LTspice, 2N2222 transistors, 1N4148 diode, 100 kΩ load)

A B	00	01	10	11
NAND V_Y	3.99 V	3.99 V	3.99 V	**0.01 V**
NOR V_Y	**3.99 V**	0.01 V	0.01 V	0.01 V



Transfer characteristic: the output switches near 1.37 V (NAND) / 1.39 V (NOR), inside TTL's undefined band of 0.8–2.0 V.

Standard TTL levels (74 series):

Parameter	Value
$V_{IL(max)}$	0.8 V
$V_{IH(min)}$	2.0 V
$V_{OL(max)}$	0.4 V
$V_{OH(min)}$	2.4 V
noise margins	NM_L = 0.8 − 0.4 = **0.4 V**; NM_H = 2.4 − 2.0 = **0.4 V**
fan-out	10
propagation delay	≈ 10 ns per gate

DO IT

LTspice (if done as simulation):

1. Draw the circuit with two NPNs for Q1 (bases tied, collectors tied, one emitter per input). Q2–Q4 = 2N2222; D = 1N4148. VCC = 5 V.
2. Inputs are DC sources of 0 or 5 V. Run `.op` for each of the four input pairs.
3. For the VTC, hold B high (NAND) or low (NOR) and run `.dc VA 0 5 0.01`.

Breadboard: the same circuit with BC547s and a 5 V supply. Inputs to 0 V / 5 V through the switches; output to an LED through about 330 Ω, or to the DMM.

WRITE IN YOUR RECORD

Result: the TTL NAND gave a HIGH output (≈ 4 V) for any LOW input and a LOW output (≈ 0 V) only when both inputs were HIGH. The TTL NOR gave a HIGH output only for 00. Both switched at ≈ 1.4 V.

Theory for the record: transistor-transistor logic uses a multi-emitter input transistor that performs the AND function. A phase-splitter transistor drives a totem-pole output: an active pull-up transistor with a diode, and an active pull-down transistor. Any LOW input diverts Q1's base current, turning off the phase splitter and the pull-down, so the output is HIGH. With all inputs HIGH, current flows through Q1's base–collector junction into the phase splitter, saturating the pull-down, so the output is LOW. A NOR uses parallel phase splitters. TTL works from 5 V, with $V_{IL} \le 0.8$ V, $V_{IH} \ge 2.0$ V, noise margin 0.4 V and fan-out 10. A floating input behaves as HIGH.

CHECK 1

(a) Why does a floating TTL input act as HIGH? (b) What would happen without the diode D? (c) Why can't two totem-pole outputs be wired together?

Answers

Opening question. A LOW input lets Q1's base current flow **out through that emitter** into the input source, which starves Q2. The output stage is a totem pole because Q4 and Q3 are stacked between VCC and ground. The noise margin is **0.4 V** each way.

Check 1. (a) With nothing pulling the emitter down, Q1's base–emitter junction has no path, just as if the input were HIGH. R1's current goes to Q2. (b) With the output LOW, Q4's base (Q2's collector at about 0.9 V) could still forward-bias Q4. Both Q4 and Q3 would conduct, wasting current and raising V_{OL}. D adds the extra voltage drop that keeps Q4 off. (c) If one output is HIGH and the other LOW, Q4 of one and Q3 of the other form a low-resistance path from VCC to ground. A large current flows and can damage them. Use open-collector gates for wired logic.

11 · Revision set: one question per idea

Answer from memory on paper, then check. A miss points back to its chapter, shown in brackets.

1. VCC and GND pins of: a 14-pin gate IC, a 16-pin MSI chip, the 7483, the 7473. *(00)*
2. What does a floating TTL input read as, and where do you tie unused AND and NOR inputs? *(00)*
3. Build OR from NANDs only, and AND from NORs only. Give the gate counts. *(Exp 1)*
4. Full adder S and C_{out}. Full subtractor D and B_o. *(Exp 2)*
5. How does one 7483 subtract, and what does $C_{out} = 0$ mean afterwards? *(Exp 2)*
6. BCD adder correction condition. *(Exp 2)*
7. Which form (SOP/POS) for NAND-only, and which for NOR-only? *(Exp 3)*
8. Binary → Gray equations; Gray → binary equations. *(Exp 4)*
9. Excess-3 of 6, and why XS-3 is self-complementing. *(Exp 4)*
10. 7485 cascade inputs for a single chip. *(Exp 5)*
11. Odd-parity generator for XYZ; what checker output 1 means. *(Exp 5)*
12. f = Σm(0,3,5,7,8,10,14) on an 8:1 MUX: the eight data inputs. *(Exp 6)*
13. 74138 as a 1:8 DEMUX: where does the data go? *(Exp 7)*
14. Why does the selected LED go dark on the NAND decoder? *(Exp 8)*
15. Race-around: cause and cure. *(Exp 9)*
16. 7474 and 7473: which edge, and which asynchronous inputs? *(Exp 10)*
17. D = ? to make a T flip-flop; S, R = ? to make a D from an SR. *(Exp 10)*
18. J and K for each stage of a 4-bit synchronous up counter. *(Exp 11)*
19. Ring vs Johnson: feedback, number of states, start state. *(Exp 11)*
20. Load 1011 serially: the register contents after each of the 4 clocks. *(Exp 12)*
21. Mealy vs Moore; what "overlapping" means for 1010. *(Exp 13)*
22. The 101 detector equations with D flip-flops. *(Exp 13)*
23. TTL NAND: the output stage's name, and why the diode is there. Noise margin. *(Exp 14)*

Answers

1. 14-pin: VCC 14, GND 7. 16-pin: 16, 8. **7483: VCC 5, GND 12. 7473: VCC 4, GND 11.**
2. HIGH. Unused AND/NAND inputs → +5 V or a used input; unused OR/NOR inputs → GND.
3. OR = NAND(NAND(A,A), NAND(B,B)), 3 gates. AND = NOR(NOR(A,A), NOR(B,B)), 3 gates.
4. $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + BC_{in} + AC_{in}$. $D = A \oplus B \oplus B_i$, $B_o = A'B + (A \oplus B)'B_i$.
5. XOR each B bit with M = 1 and set $C_{in} = M = 1$, giving A + B' + 1. $C_{out} = 0$ means A < B; the result is in 2's complement.
6. $K = C_4 + S_3S_2 + S_3S_1$. When K = 1, add 0110.
7. NAND-only: SOP. NOR-only: POS.
8. $G3 = B3$, $G_i = B_{i+1} \oplus B_i$. $B3 = G3$, $B_i = B_{i+1} \oplus G_i$.
9. 6 + 3 = 9 = **1001**. Inverting XS-3(d) gives XS-3(9 − d).
10. I(A=B) = 1, I(A>B) = I(A<B) = 0.
11. $P = (X \oplus Y \oplus Z)'$. C = 1 means an even number of 1s arrived: an error.
12. Select ABC: I0 = D', I1 = D, I2 = D, I3 = D, I4 = D', I5 = D', I6 = 0, I7 = D'.
13. Into E1' (active LOW), with E2' = 0 and E3 = 1. The selected output follows D.
14. The outputs are active-LOW: the selected one goes to 0.
15. A level-triggered JK with J = K = 1 toggles repeatedly while CLK = 1. Cure: edge triggering or master-slave.
16. 7474: positive edge, PR' and CLR'. 7473: negative edge, CLR' only.
17. $D = T \oplus Q$. S = D, R = D'.
18. $J_A = K_A = 1$; $J_B = K_B = Q_A$; $J_C = K_C = Q_AQ_B$; $J_D = K_D = Q_AQ_BQ_C$.
19. Ring: Q0 → D3, n states, must be preset to 1000. Johnson: Q0' → D3, 2n states, starts from 0000.
20. 1000, 1100, 0110, 1011.
21. Mealy: output depends on state and input. Moore: output depends on state only. Overlapping: after a match, go to the state for "10", not the start.
22. $D_1 = Q_0X'$, $D_0 = X$, $Z = Q_1X$.
23. Totem pole. The diode keeps Q4 off when the output is LOW. Noise margin 0.4 V.